

Decaboard: Introduction to Shader Programming

GPUs and Shaders

Modern computers have powerful graphics processing units called *GPUs*. A single GPU can have thousands of *cores*, i.e. small CPUs designed specifically for GPU can have thousands of *cores*, i.e. small CPUs designed specifically for graphics operations.

One of the things they support is [shader programming](#), a style of programming designed specifically for computer graphics.

For example, the NVIDIA GeForce RTX 4090 GPU has 16,384 cores that can run in parallel. You could use it to compute lighting effects (shading) to a scene in *real time*. Doing the calculations needed for shading in real time is impressive: the computers main CPU is not fast enough to apply graphics operations to an entire scene in *real time*.

But programming thousands of cores presents a problem. How do you tell them what to do? How do you divide a problem into 16,000 sub-problems? The answer given by shaders is surprisingly simple: you, the programmer, write a *single function* that sets the color of a *single pixel*, and then that function is applied to all the pixels in an image in parallel.

At first this might seem *too simple*. How can you do anything useful by writing *one* function that sets the color of *one* pixel? The input to this function is typically the row and column of the pixel, and the time that has elapsed since the program started. That's enough information to computer wide variety of things, including shapes, animation, lighting effects, and much more.

It takes some practice to get the hang of using shaders. Some things that are easy to do in regular programming might require some cleverness to implement as a shader.

Shaders are typically written in a **shader language** designed specifically for the task. For instance, [GLSL](#) is a popular C-like language for writing shaders. If you are interested in learning more, [The Shader Book](#) is a good start that has many interactive examples. [ShaderToy](#) is another great resource that shows many amazing examples of what shaders can do.

Decaboard: An Introduction to Shader Programming

Decaboard introduces shader-style programming using a 10 x 10 grid of squares. Instead of setting colors as you would with a GPU shader, you set the angle of each square. You write one function that is applied to all the squares at the same time.

To try decaboard, you first need Python installed on your computer with turtle graphics working. Then download [decaboard.py](#) and [example.py](#) to the same directory, and run [example.py](#). A window of 100 rotated squares should pop up.

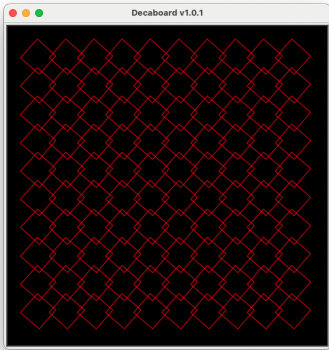
[example.py](#) looks like this:

```
# example.py
```

```
import decaboard

def angleIt(row, col, elapsed_seconds):
    return 41

decaboard.run_board(angleIt)
```



All decaboard programs have this same structure: the programmer only writes the `angleIt` function. The `angleIt` function is passed to `decaboard` and applied to all 100 squares.

Decaboard does not actually use a GPU shader, it just simulates the shader style programming using regular Python.

To learn more, try these problems:

- [Problem Set 1](#). In these problems you're given a picture of the board, write `angleIt` to make that picture.
- [Problem Set 2](#). These problems give you an `angleIt` function, and ask you to draw the picture it makes. These show how to animate the squares, and how even simple functions can make interesting and unexpected patterns. Techniques from the problem sets can be combined to make more complex patterns.